Agenda

1. Introduction to Recursion
2. Function Call Tracing
3. Factorial of N
4. Printing numbers in Inc Order
5. Printing numbers in Dec Order
6. Time & Space Complexity
7. Fibonacci

Why Recursion ?

- Pre-requisite of Backtracking , Trees , D.P , Graphs
- Sorting Algorithm's [Merge , Sort , Quick Sort]

Using recursion a problem can be broken into smaller problem (subproblem) and the solution is generated using the subproblems.

Recursion Definition → Function calling itself.

Steps of writing recursive code

Function Definition - Decide exactly what the function will do for the given problem

Main Logic - Break problem down into smaller subproblems to solve the main problem

Base Condition - Identify the smallest input for which we already know the answer and we should stop

EXAMPLE: Sum of first N natural numbers → $\frac{N*(N+1)}{2}$

sum(5) = 1 + 2 + 3 + 4 + 5

sum(4)

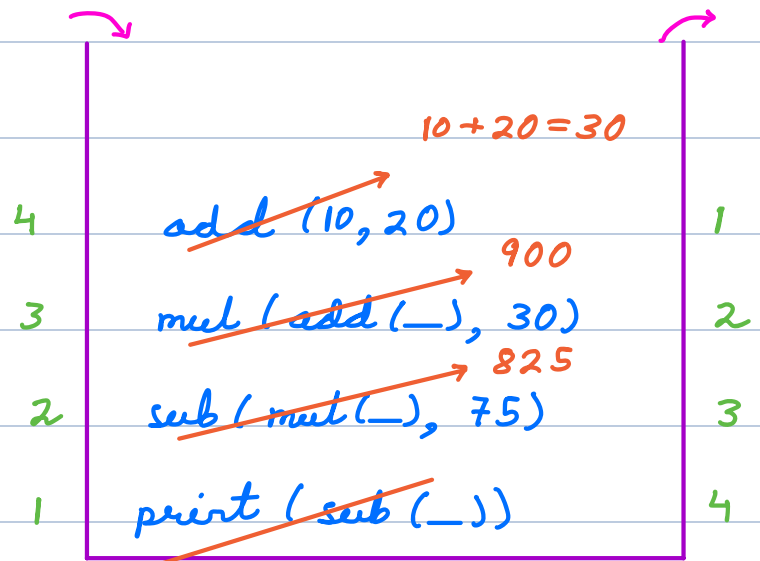
sum(N) = sum(N-1) + N

```
int sum(N) {
    if (N == 1) return 1
    return sum(N-1) + N
}
```

Function Call Tracing

```
int add ( int x, int y ){
    return x + y ;
}
int mul ( int n, int y ){
    return x * y ;
}
int sub ( int x, int y ){
    return x - y ;
}
void print ( int x ){
    print (x) ;
}
```

```
function main() {
    int x = 10, y = 20;
    print( sub( mul( add( x, y ), 30), 75));
}
```



o/p → 825

stack → Last In First Out

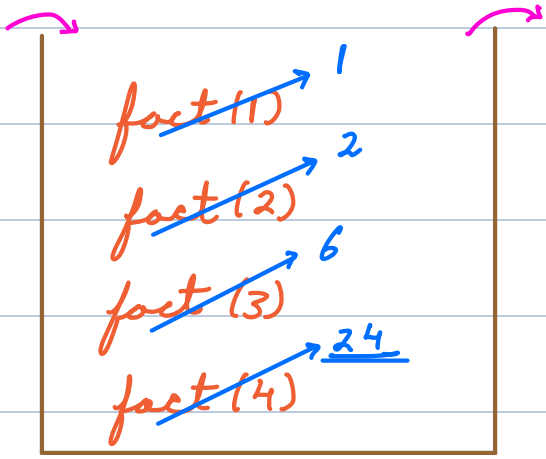
N Factorial

$fact(N) = N * (N-1) * (N-2) \ldots 1$

$fact(5) = 5 * \underbrace{4 * 3 * 2 * 1}_{fact(4)} = 120$

$fact(N) = N * fact(N-1)$

```
long fact(N) {
    if (N <= 1) return 1
    return N * fact(N-1)
}
```




```
fact(4) {
    return 4 * fact(3) {        // 4*6 = 24
        return 3 * fact(2) {    // 3*2 = 6
            return 2 * fact(1) {  // 2*1 = 2
                return 1
            }
        }
    }
}
```

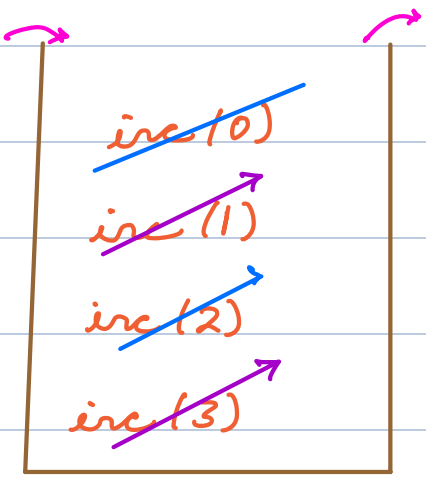
Print numbers in increasing order from 1 to N

inc (3) → o/p → 1 2 3

inc (2)

inc(N) → o/p: inc(N-1) N

```
void inc (N) {
    if (N == 0) return
    inc (N-1)
    print (N)
}
```




```
inc (3) {
    inc(2) {
        inc(1) {
            inc(0) { }
            print (1) ✓
        }
    print (2) ✓
```

}

print(3) ✓

}

Whirlpool

Problem

Whirlpool wants to design a timer for their **washing machines**. This feature is a simple countdown timer. When a user sets a time, for example, 10 minutes, the washing machine needs to show each minute passing, counting down until it reaches 0.

Your task is to write a program that takes an integer **A** (the time in minutes set by the user) and then prints out each minute as it counts down to **0**.The requirement is that after a user set a timer for the washing machine for some time say **A** , the washing machine should display each minute after that decremented one by one till the time becomes **0**.

dec(N) → N*(N-1)*...1

```
void dec(N) {          // N = 3
    if (N == 0) return
    print(N)
    dec(N-1)
}
```

```
dec(3) {
    print(3) ✓
    dec(2) {
        print(2) ✓
        dec(1) {
            print(1) ✓
            dec(0) { }
        }
    }
}
```

Recursion - 1

SCALER

Time Complexity

```
Time Complexity = O(Number of function calls * Time per function call)
```

O(N) * O(1) → O(N)

```
int fact (int N){
    if (N==0) { return 1 }
    return fact(N-1) * N;
}
```

TC = O(N)

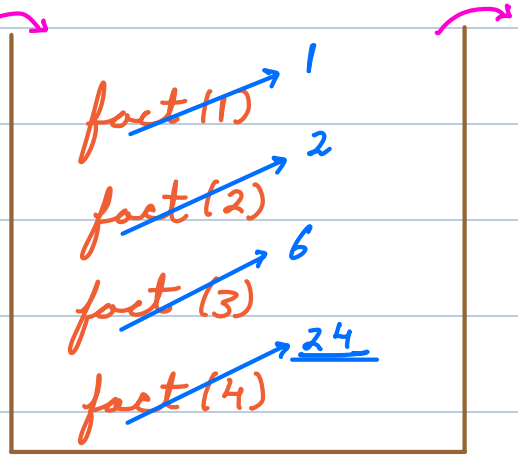
Space Complexity = O (Recursive stack space + SC of other DS used)

Max size of stack at any point.

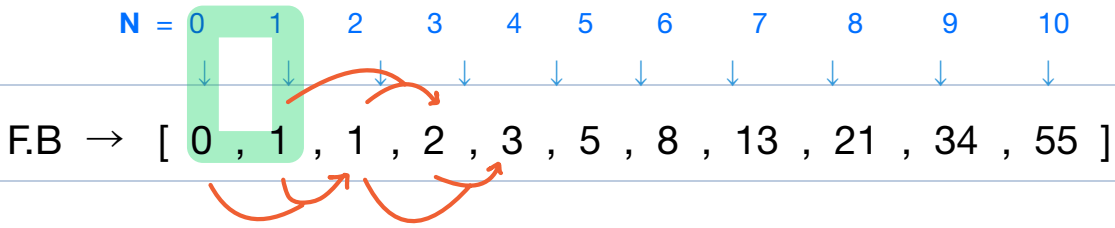
```
long fact (N) {
    if ( N <= 1 ) return 1
    return N * fact (N-1)
}
```

4

SC = O(N)



Nth Fibonacci



Given value of N. Write a recursive function of find Nth fibonacci number.

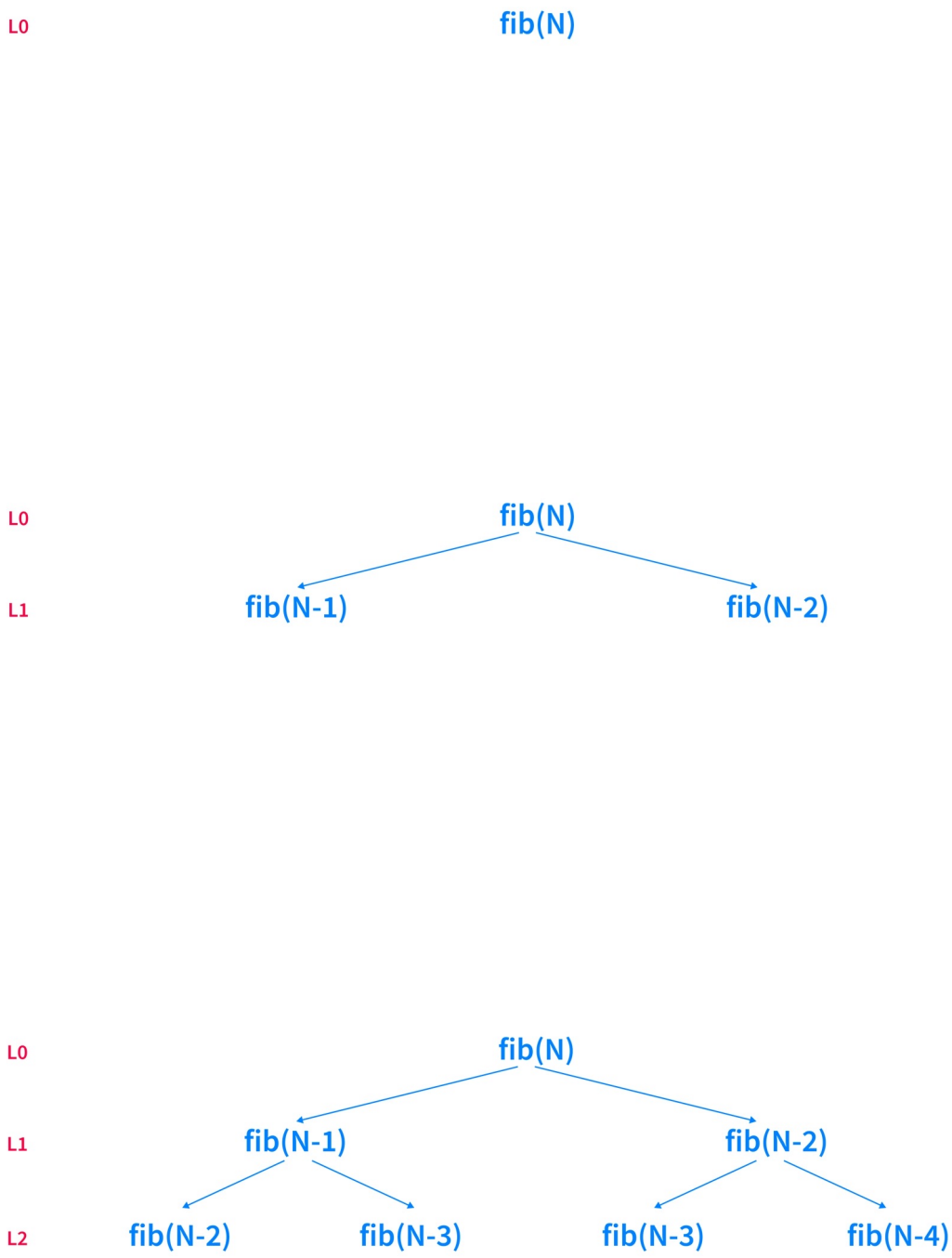
```
int fib(N) {
    if (N <= 1)  return N
    return fib(N-1) + fib(N-2)
}
```

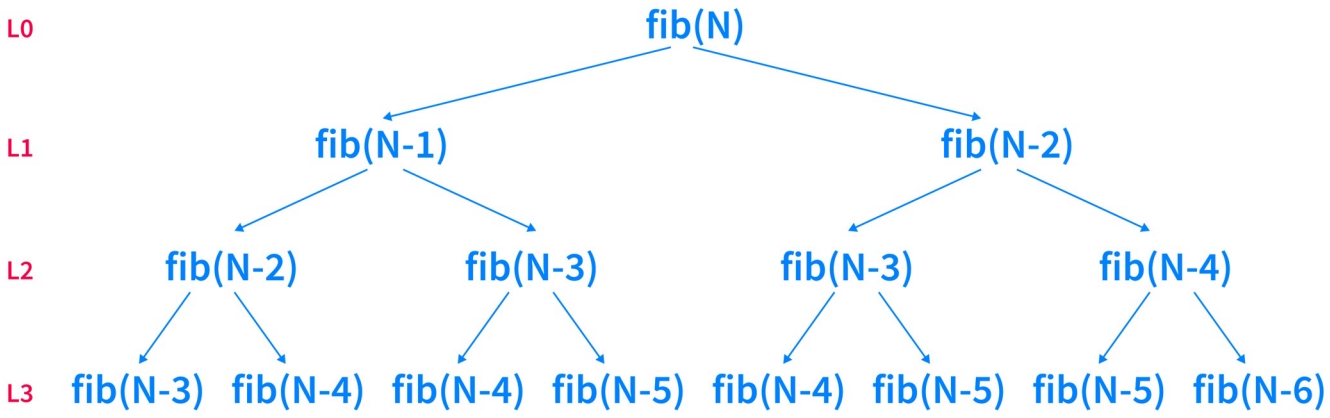
(1) (3) (2)

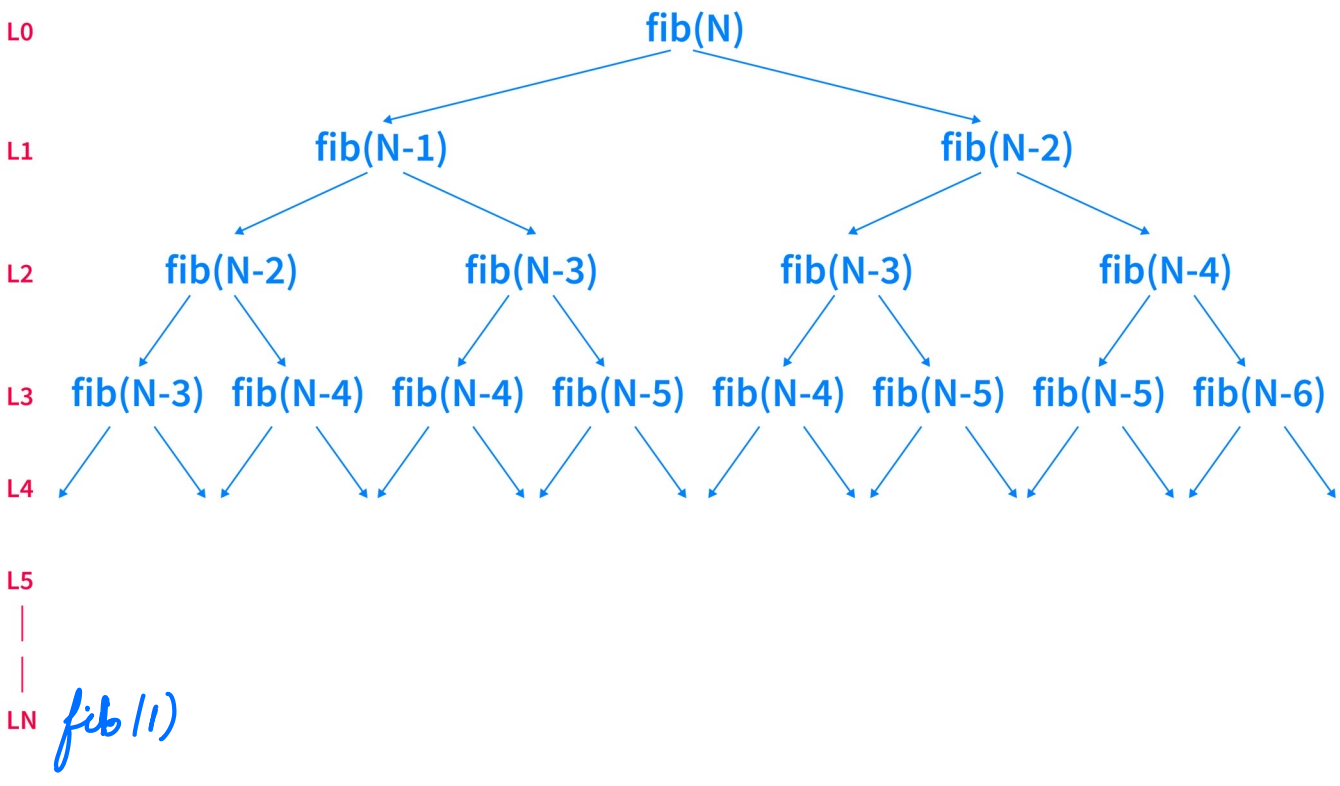
```
fib(3) {
    fib(2) {
        fib(1) { return 1 }
        fib(0) { return 0 }
        return 1+0 = 1
    }
    fib(1) { return 1 }
    return 1+1 = 2
}
```

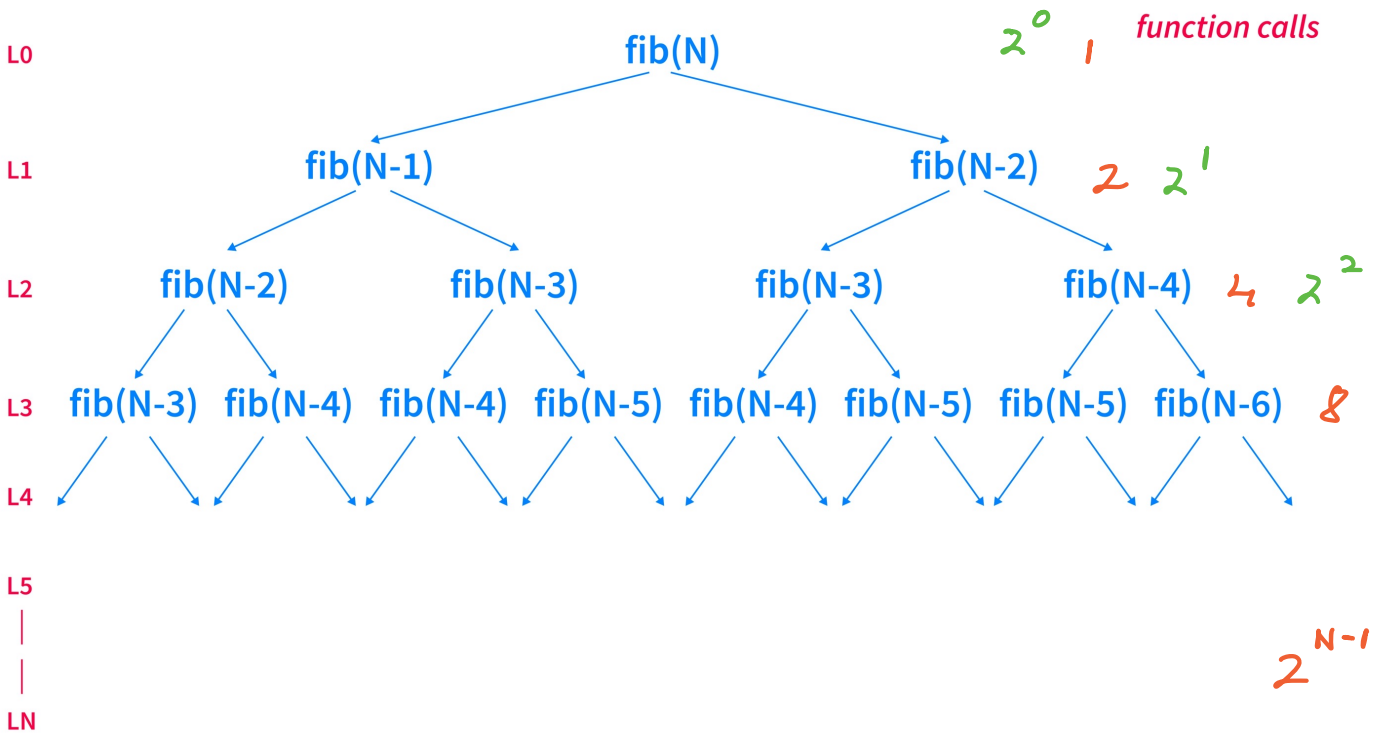
T.C Fibonacci

```
int fib(int N){
    if(N ≤ 1) {return N}
    return fib(N-1) + fib(N-2);
}
```




L0
fib(N)
L1
fib(N-1)
fib(N-2)
L2
fib(N-2)
fib(N-3)
fib(N-3)
fib(N-4)
L3
fib(N-3)
fib(N-4)
fib(N-4)
fib(N-5)
fib(N-4)
fib(N-5)
fib(N-5)
fib(N-6)


L0
fib(N)
L1
fib(N-1)
fib(N-2)
L2
fib(N-2)
fib(N-3)
fib(N-3)
fib(N-4)
L3
fib(N-3)
fib(N-4)
fib(N-4)
fib(N-5)
fib(N-4)
fib(N-5)
fib(N-5)
fib(N-6)
L4
L5
LN
fib(1)



total function calls : $1 + 2 + 4 + 8 + \ldots 2^{N-1}$

$$= \frac{1(2^N - 1)}{2 - 1} = 2^N - 1$$

$TC = O(2^N)$

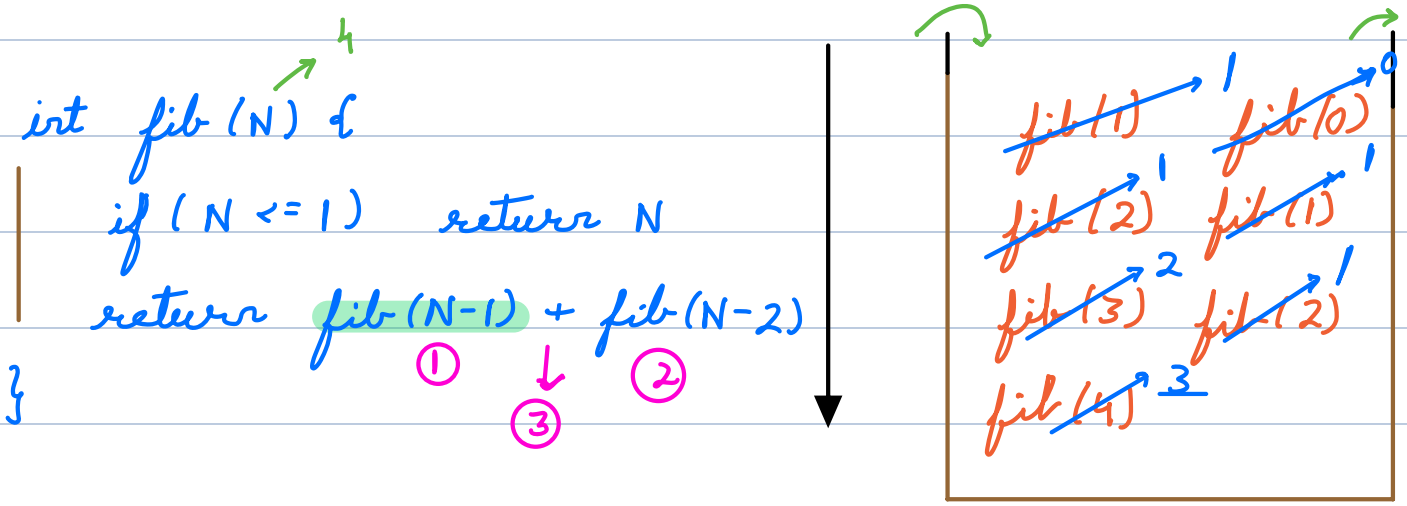
S.C Fibonacci

Space complexity is the maximum amount of stack space used by the algorithm.

When a function is called, it is added to the stack.
When a function returns, it is popped off the stack.

We're not adding all of the function calls to the stack at once.
We only make n calls at any given time as we move up and down branches.

We proceed branch by branch, making our function calls until our base case is met, then we return and make our calls down to the next branch.



TC = $O(2^N)$ SC = $O(N)$